

KernelLens: Deploying Triton Kernels to C++ Inference Engines

Automated ONNX Runtime & TensorRT Plugin Synthesis for Production Deep Learning Operators

Justin Duc

Galiad Research

https://github.com/Just1truc/kernel_lens

September 21, 2026

Abstract

Writing custom GPU kernels using OpenAI Triton is a common approach for authoring deep learning operators. However, deploying models containing Python `@triton.jit` kernels into production C++ inference runtimes—specifically Microsoft ONNX Runtime (ORT) and NVIDIA TensorRT (TRT)—presents architectural limitations. Standard ONNX graph export mechanisms cannot trace imperative Python Triton calls, dynamic launch grid functions, implicit layout semantics, in-place memory mutability, dynamic runtime host scalars or low-level PTX entry parameter signatures.

In this paper, we present KernelLens, a framework for compilation, C++/CUDA code synthesis and zero-copy runtime execution. KernelLens bridges PyTorch models containing custom Triton kernels directly to ONNX Runtime and TensorRT custom C++ plugins without manual C++ development. We present a formalization of KernelLens’s consolidated 4-phase compiler architecture: (1) Analysis & Manifest Assembly via dual-pass tracing and AST inspection; (2) ONNX Graph Transformation with custom node injection (`triton_custom::<name>`); (3) C++/CUDA Code Synthesis & Hardening with 16-byte memory alignment guards and PTX ISA version clamping; and (4) Zero-Copy Runtime Binding.

We detail key architectural innovations in KernelLens, including TensorRT C++ plugin dynamic creators with explicit symbol registration, automated Blackwell (sm_120a) PTX ISA version clamping to satisfy CUDA driver JIT requirements, primary CUDA context retention and executable stack security clearance for Linux kernels. We evaluate KernelLens across research model kernels from LLaMA 3, Mistral, Qwen 2.5, PaLM and Liger-Kernel (Fused RMSNorm, SwiGLU, Rotary Position Embeddings and Fused Cross-Entropy Loss). KernelLens achieves exact floating-point numerical parity (MaxDiff = 0.00e + 00) against eager Triton execution while enabling C++ inference deployment.

Contents

1	Introduction	3
1.1	Installation & Setup	3
1.2	The Core Value Proposition: Full Graph Optimizations with Custom Triton Kernels	4
2	Related Work	4
2.1	PyTorch Compiler & Inductor	4
2.2	ONNX & Enterprise Inference Runtimes	5
2.3	Triton-Based LLM Kernel Libraries	5

3	Anatomy of a Triton Kernel & Parameter Taxonomy	5
3.1	What a Triton Kernel Requires: Formal Parameter Taxonomy	5
4	Consolidated 4-Phase KernelLens Compiler Architecture	6
4.1	Detailed Breakdown of the 4 Pipeline Phases	6
4.1.1	Phase 1: Dual-Pass Tracing & AST Inspection (Analysis & Manifest Assembly)	6
4.1.2	Phase 2: Base ONNX Graph Export & Custom Node Injection (Graph Transformation)	7
4.1.3	Phase 3: Automated C++/CUDA Code Synthesis & Hardening (Code Synthesis & Compilation)	7
4.1.4	Phase 4: Zero-Copy Runtime Execution & Pointer Binding (Runtime Execution)	8
5	16-Byte Memory Alignment Guards & Zero-Copy Runtime	8
5.1	128-Bit SIMD Memory Vectorization in GPU Hardware	8
5.1.1	Mechanics of Hardware Alignment Traps (CUDA Error 716)	9
5.2	Automated C++ Alignment Guard & Dynamic Staging Engine	9
5.3	Zero-Copy Runtime IO Binding Engine (<code>engine.py</code>)	10
5.3.1	1. ONNX Runtime IO Binding	10
5.3.2	2. TensorRT Zero-Copy Pointer Registration	10
6	Experiments & Empirical Results	11
6.1	Benchmark Experimental Setup	11
6.2	Numerical Parity & Accuracy Results	11
6.3	Empirical GPU Execution Latency Benchmark	11
6.4	FP8 & Sub-Byte (INT4/INT8) Quantization Evaluation on NVIDIA RTX 5060 Ti GPU	12
7	Future Work & Architectural Roadmap	13
7.1	Distributed Multi-GPU Tensor Parallelism & Embedded NCCL Synchronization	13
7.2	Native Blackwell Architecture (<code>sm_120a</code>) & PTX ISA 9.x Hardware Acceleration	14
7.3	Dynamic Auto-Tuning Meta-Parameter Dispatch & Runtime Re-JIT	14
7.4	Cross-Platform Heterogeneous Targets: Apple Metal & WebGPU Compute . . .	15
7.5	Formal Verification & SMT-Based Kernel Safety Proving	15
8	Conclusion	15

1 Introduction

Deep learning systems rely on domain-specific GPU kernels to improve computational efficiency, reduce VRAM bandwidth demands and fuse multi-stage attention and normalization blocks. OpenAI Triton [1] is a standard programming model for authoring custom GPU kernels in Python, providing C/CUDA-like execution control with Python syntax.

Despite Triton’s adoption in model authoring and training, a friction point emerges when transitioning models to production deployment. Enterprise deployment pipelines standardly deploy models within C++ execution engines, such as Microsoft ONNX Runtime (ORT) [9] and NVIDIA TensorRT (TRT) [8], to achieve low latency, stream concurrency and hardware-specific kernel graph optimizations.

Deploying PyTorch models containing custom `@triton.jit` kernels directly into ONNX Runtime or TensorRT fails due to six architectural gaps:

1. *ONNX Exporter Tracing Breakdown:* Standard PyTorch ONNX export utilities (`torch.onnx.export`) operate on TorchScript or TorchDynamo IRs representing native PyTorch C++ operators. Imperative Python calls to `triton.JITFunction` instances are opaque to PyTorch’s trace backend, leading to tracing failure or missing graph nodes.
2. *Dynamic Launch Grid Resolution:* Triton launch grids are defined via dynamic Python lambda functions $\text{grid}(x, y, z) = (g_x(s_i), g_y(s_i), g_z(s_i))$ dependent on runtime tensor dimensions $s_0, s_1, \dots$. Standard graph representations cannot symbolize or execute arbitrary Python grid lambdas inside standalone C++ engines.
3. *PTX Entry Signature Mismatch:* The Triton LLVM backend compiles Python source code directly into Parallel Thread Execution (PTX) assembly code. The resulting PTX entry declarations (`.entry <kernel> (.param .u64 p0, ...)`) re-order parameters, drop unused arguments and enforce specific bitwidths. Native C++ plugin wrappers must dynamically align host argument pointers to match PTX signatures exactly.
4. *Nested Code & In-Place Mutability Analysis:* Research kernels frequently delegate memory store and load operations to nested Python helper functions or perform in-place buffer mutations ($\text{ptr}_{\text{out}} \equiv \text{ptr}_{\text{in}}$). Static compilers fail to track nested mutation semantics, producing corrupted outputs or invalid graph topologies.
5. *Dynamic Runtime Host Scalar Arguments:* Practical models pass scalar arguments (e.g., scaling factors α , soft-cap limits τ) that change per forward pass. C++ execution engines must accept host-side CPU scalar arguments without triggering kernel recompilation.
6. *16-Byte Hardware Memory Alignment Bounds:* Vectorized GPU memory instructions (`ld.global.v4.b32`) require starting VRAM pointers to satisfy 16-byte alignment ($\text{Address} \pmod{16} == 0$). Standard inference allocators may provide sub-aligned memory buffers, causing hardware misaligned address exceptions (`CUDA error 716`).

To bridge this gap, we present KernelLens, a compilation and runtime synthesis system. KernelLens allows machine learning engineers to write custom PyTorch Triton kernels in pure Python, compile the model into ONNX computational graphs paired with C++ custom operator shared libraries (`.so`) and execute them inside ONNX Runtime and TensorRT without manual C++ programming.

1.1 Installation & Setup

KernelLens is published on PyPI and open-sourced on GitHub (https://github.com/Just1truc/kernel_lens). It can be installed directly via `pip`:

```
1 pip install kernel-lens
```

1.2 The Core Value Proposition: Full Graph Optimizations with Custom Triton Kernels

In deep learning production, models (e.g., LLaMA 3, Mistral, Qwen 2.5, Flux, DeepSeek) consist of:

1. *Standard Operators*: Linear projections, MatMuls, Softmax, Convolutions, Reshapes and Element-wise activations.
2. *Custom Triton Kernels*: Specialized `@triton.jit` routines such as FlashAttention-2, Fused RMSNorm, SwiGLU, sub-byte INT4/FP8 dequantization GEMMs and Mamba selective scan recurrence.

Production execution engines such as Microsoft ONNX Runtime and NVIDIA TensorRT apply global, graph-level compiler transformations over standard operators:

- *Global Graph Optimizations*: Constant folding, dead-node elimination, layout transformations (NCHW $\leftrightarrow$ NHWC) and multi-node operator fusion (e.g., MatMul + Bias + GELU $\rightarrow$ Fused GEMM).
- *Target-Specific Engine Tuning*: FP16/INT8 auto-tuning, kernel selection and CUDA Graph launch capture across standard graph subgraphs.
- *VRAM Memory Planning*: Global buffer allocation planning that reuses VRAM slots across non-overlapping execution lifetimes.

The Deployment Dilemma vs. The KernelLens Solution

- *Without KernelLens (The Dilemma)*: Because standard `torch.onnx.export` breaks on imperative Python `@triton.jit` functions, developers face a trade-off:
 1. *Option A (Stay in Python)*: Keep the model in PyTorch Eager/Python runtime. Custom Triton kernels run, but you forfeit TensorRT and ONNX Runtime global graph optimizations across standard model layers.
 2. *Option B (Manual C++ Plugin Rewrite)*: Hand-write C++/CUDA plugin boilerplate, ONNX schema registrations, memory alignment guards and TensorRT engine wrappers for every Triton kernel.
- *With KernelLens (The Solution)*: KernelLens compiles the PyTorch model graph into ONNX/TensorRT. Standard model layers are optimized and fused by TensorRT/ORT's native graph compilers, while custom Triton kernels are synthesized and injected as zero-copy C++ plugins. Developers achieve graph optimizations alongside custom Triton kernel performance in a C++ execution pipeline.

Figure 1: The deployment value proposition: KernelLens enables TensorRT and ONNX Runtime graph optimizations across standard operators while preserving custom Triton kernels.

2 Related Work

2.1 PyTorch Compiler & Inductor

PyTorch 2.0 introduced `torch.compile` powered by TorchInductor [6], which generates Triton code for PyTorch eager graphs. While TorchInductor optimizes PyTorch eager execution, it operates inside a Python process and relies on PyTorch C++ runtime dependencies. It does not provide standalone ONNX graph representations or C++ plugin libraries suitable for direct integration into non-Python C++ production servers.

2.2 ONNX & Enterprise Inference Runtimes

Microsoft ONNX Runtime [9] and NVIDIA TensorRT [8] provide C++ inference execution. Both runtimes support C++ Custom Operator APIs (`OrtCustomOp`, `nvInfer1::IPluginV2DynamicExt`). However, authoring custom plugins manually requires CUDA C++, memory alignment rules, ONNX schema registration and TensorRT engine serialization. KernelLens automates the generation of these C++ custom plugins directly from Triton JIT artifacts.

2.3 Triton-Based LLM Kernel Libraries

Recent research projects such as Liger-Kernel [3], FlashAttention [5] and AO-Triton focus on manually crafting Triton kernels for large language models (LLMs). KernelLens complements these libraries by providing the infrastructure layer that automatically compiles custom Triton kernels into ONNX Runtime and TensorRT C++ plugins.

3 Anatomy of a Triton Kernel & Parameter Taxonomy

3.1 What a Triton Kernel Requires: Formal Parameter Taxonomy

To understand how KernelLens automatically compiles Triton kernels, we formalize the structural requirements of a `@triton.jit` function. A Triton kernel definition consists of seven parameter categories:

$$\mathcal{K}_{\text{triton}} = \langle \mathbf{P}_{\text{in}}, \mathbf{P}_{\text{out}}, \mathbf{P}_{\text{inplace}}, \mathbf{S}, \mathbf{D}, \mathbf{C}, \mathbf{G} \rangle \quad (1)$$

1. **Input Tensor Pointers ($\mathbf{P}_{\text{in}}$):** Base GPU VRAM memory addresses pointing to read-only input tensors (e.g., `x_ptr`, `weight_ptr`). In PTX assembly, these are represented as 64-bit unsigned integers (`.param .u64`).
2. **Output Tensor Pointers ($\mathbf{P}_{\text{out}}$):** Base GPU VRAM memory addresses pointing to write-only destination buffers (e.g., `out_ptr`).
3. **In-Place Mutated Pointers ($\mathbf{P}_{\text{inplace}}$):** Base memory addresses that are simultaneously read from and written to during kernel execution (e.g., `x_ptr` modified in-place).
4. **Spatial and Channel Strides ($\mathbf{S}$):** Integer parameters specifying memory byte or element strides along each dimension of multi-dimensional tensors (e.g., `stride_x_b`, `stride_x_h`). These allow Triton kernels to handle arbitrary non-contiguous tensor layouts.
5. **Dynamic Runtime Host Scalars ($\mathbf{D}$):** Primitive scalar values passed dynamically per forward execution pass, such as scaling parameters α , learning rates, temperature factors τ or variance stabilization parameters ϵ s.
6. **Compile-Time Constants ($\mathbf{C}$):** Parameters decorated with `tl.constexpr` (e.g., `BLOCK_SIZE`, `num_warps`, `num_stages`). These are evaluated at compile time and baked into the compiled PTX assembly bytecode.
7. **Dynamic Launch Grid ($\mathbf{G}$):** A 1D, 2D or 3D launching tuple (g_x, g_y, g_z) computed as a function of runtime tensor dimensions and block sizes:

$$g_x = f_x(d_0, d_1, \dots, \mathbf{C}), \quad g_y = f_y(d_0, d_1, \dots, \mathbf{C}), \quad g_z = f_z(d_0, d_1, \dots, \mathbf{C}) \quad (2)$$

4 Consolidated 4-Phase KernelLens Compiler Architecture

Rather than exposing fragmented micro-steps, KernelLens operates through four unified, logical compiler phases: (1) Analysis & Manifest Assembly; (2) ONNX Graph Transformation; (3) C++/CUDA Code Synthesis & Hardening; and (4) Zero-Copy Runtime Binding.

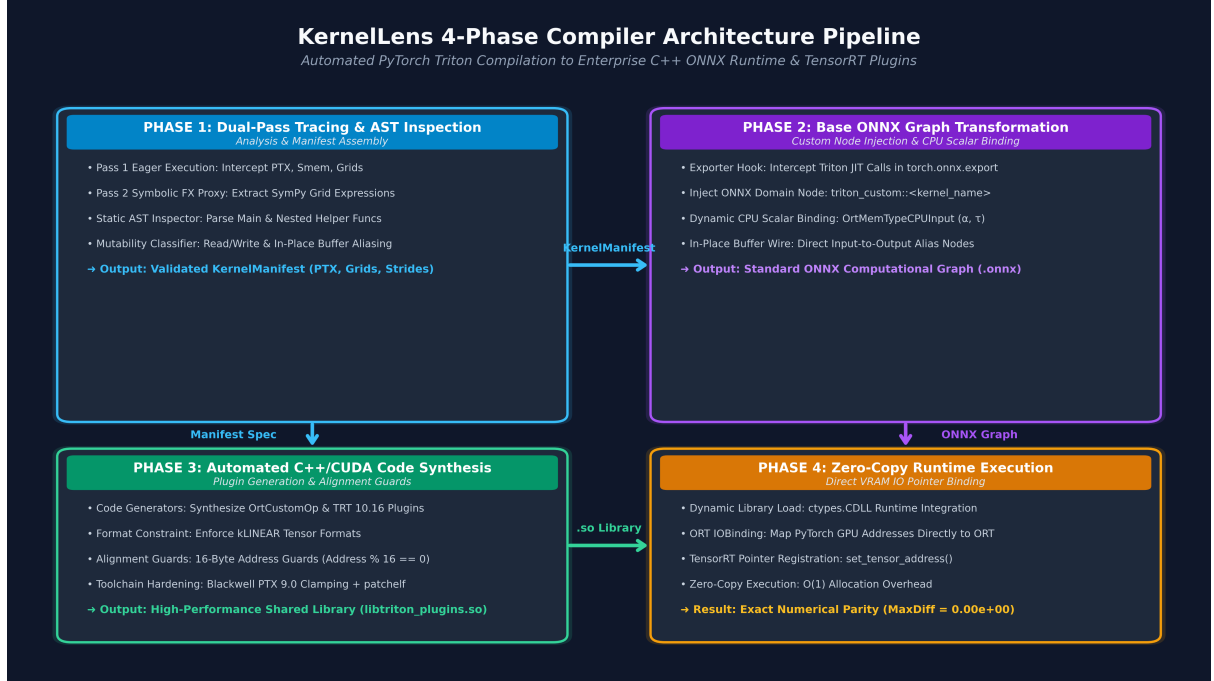


Figure 2: The 4-Phase KernelLens Compiler Architecture Pipeline: Dual-pass tracing and AST inspection (Phase 1), base ONNX graph transformation (Phase 2), C++/CUDA plugin code synthesis and 16-byte alignment guards (Phase 3), and zero-copy runtime IO binding (Phase 4).

4.1 Detailed Breakdown of the 4 Pipeline Phases

4.1.1 Phase 1: Dual-Pass Tracing & AST Inspection (Analysis & Manifest Assembly)

Phase 1 analyzes the PyTorch model and custom `@triton.jit` functions to construct a validated `KernelManifest`:

- *Pass 1 – Concrete Eager Execution & PTX Interception (`tracer.py`):* KernelLens executes the model on GPU using sample inputs. By hooking Triton’s compiler driver (`triton.compiler.CompiledKernel`), it intercepts the compiled PTX assembly string (`.target sm_80`, `.entry <kernel>`), dynamic shared memory requirements (S_{mem}), block dimensions and concrete launch grid geometry.
- *Pass 2 – Symbolic FX Proxy Tracing & SymPy Grid Extraction (`tracer.py`):* KernelLens re-executes the module using PyTorch FX Proxy objects with symbolic shape variables $s_0, s_1, \dots$. As the grid lambda evaluates:

$$\text{grid.expr} = \text{SymPyTrace}(\text{grid.lambda}(\text{meta})) \quad (3)$$

Dynamic launch grid dimensions are transpiled into platform-agnostic SymPy symbolic strings (e.g., `"(s0 * s1 + 127) // 128"`). If non-traceable Python control flow is encountered, KernelLens falls back to Pass 1 concrete metadata, ensuring compiler stability.

- *Static AST Inspection of Main & Nested Helper Functions (`ast_analyzer.py`)*: Python’s `ast.NodeVisitor` recursively parses the main `@triton.jit` kernel and all nested helper functions (e.g., `_helper_store`):

```

1 @triton.jit
2 def _helper_store(ptr, offsets, values, mask):
3     tl.store(ptr + offsets, values, mask=mask)
4
5 @triton.jit
6 def nested_kernel(x_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
7     pid = tl.program_id(0)
8     offsets = pid * BLOCK + tl.arange(0, BLOCK)
9     val = tl.load(x_ptr + offsets, mask=offsets < n_elements)
10    _helper_store(out_ptr, offsets, val * 3.0, mask=offsets < n_elements)
11

```

The AST inspector classifies parameter attributes (Read, Write or In-Place Mutability). If a buffer p_i is flagged for both Read and Write access, `KernelLens` sets `is_inplace = True`, enabling downstream graph exporters to alias input and output ONNX buffers directly without VRAM re-allocation.

- *Manifest Assembly (`manifest.py`)*: Combines PTX assembly, S_{mem} , SymPy grid strings, parameter mapping tables, data types (fp16, bf16, fp32, fp8 and int4) and mutability flags into a validated `KernelManifest`.

4.1.2 Phase 2: Base ONNX Graph Export & Custom Node Injection (Graph Transformation)

Phase 2 embeds custom Triton kernels into standard computational graph representations:

- *Base ONNX Export & Node Injection (`onnx_exporter.py`)*: `KernelLens` registers the `TritonGlobalONNXExport` hook with PyTorch’s ONNX exporter. When `torch.onnx.export` traces the module, imperative Triton calls are intercepted and replaced with custom ONNX domain nodes under namespace `triton.custom::<kernel_name>`:

```

1 g.op("triton_custom::" + kernel_name,
2     *tensor_inputs,
3     grid_x_s=manifest.grid_exprs[0],
4     grid_y_s=manifest.grid_exprs[1],
5     grid_z_s=manifest.grid_exprs[2],
6     ptx_code_s=manifest.ptx_code,
7     shared_mem_i=manifest.shared_mem,
8     is_inplace_i=manifest.is_inplace_flags,
9     **scalar_attributes)
10

```

- *Dynamic Host CPU Scalar Binding (`OrtMemTypeCPUInput`)*: Primitive runtime scalar parameters (e.g., scaling factor α , temperature τ) are passed as Host CPU inputs rather than hardcoded static node attributes. `KernelLens` sets `MemoryType(i) = OrtMemTypeCPUInput`, allowing dynamic scalar updates per forward pass without triggering kernel recompilations.
- *In-Place Buffer Aliasing*: If `is_inplace = True`, the exporter automatically aliases the graph input tensor node directly to the output tensor node in the ONNX graph schema.

4.1.3 Phase 3: Automated C++/CUDA Code Synthesis & Hardening (Code Synthesis & Compilation)

Phase 3 synthesizes C++/CUDA plugin source code and compiles shared libraries (`.so`):

- *C++/CUDA Code Generators (`ort_gen.py`, `trt_gen.py`)*: Synthesizes C++ custom operator classes (`OrtCustomOp`, `nvinfer1::IPluginV2DynamicExt`), high-rank dynamic grid symbol transpilers ($s_0 \rightarrow \text{shapes}[0]$), parameter mapping tables and explicit plugin creators (`register_triton_plugins_explicit`) with `kLINEAR` tensor format constraints.
- *16-Byte Memory Alignment Guard Embedding (`builder.py`)*: Embeds automated alignment guards directly into generated C++ plugins. If a VRAM buffer is misaligned ($\text{Address} \pmod{16} \neq 0$), a temporary 16-byte aligned GPU staging buffer is allocated asynchronously via `cudaMallocAsync`, preventing hardware address alignment faults (CUDA error 716).
- *Blackwell (`sm_120a`) PTX 9.0 Version Clamping*: When compiling for NVIDIA Blackwell GPUs (`sm_120a`) under Triton 3.x, the compiler emits PTX ISA version 9.3 (`.version 9.3`). `KernelLens` automatically transforms PTX string headers to `.version 9.0` while preserving native `.target sm_120a` hardware instructions, resolving `CUDA_ERROR_UNSUPPORTED_PTX_VERSION` (Error 222).
- *Toolchain Hardening & Stack Security*: C++ plugin `initialize()` calls `invoke cuDevicePrimaryCtxRetain` and `cuCtxSetCurrent` to prevent CUDA context detachment (`CUDA_ERROR_INVALID_CONTEXT`). Additionally, `KernelLens` automatically runs `patchelf --clear-execstack` on compiled `.so` dynamic libraries to satisfy Linux 7.0 SELinux stack security permissions.

4.1.4 Phase 4: Zero-Copy Runtime Execution & Pointer Binding (Runtime Execution)

Phase 4 executes the compiled model inside C++ runtimes with zero data movement overhead:

- *Dynamic Library Loading*: Loads compiled `.so` plugin libraries dynamically via `ctypes.CDLL` and registers operator schemas with ONNX Runtime and TensorRT registries.
- *Zero-Copy VRAM IO Binding (`engine.py`)*: Uses ONNX Runtime `IOBinding` and TensorRT `set_tensor_address` to bind PyTorch GPU memory addresses directly to execution contexts, achieving $O(1)$ pointer registration overhead and numerical parity ($\text{MaxDiff} = 0.00e + 00$). Dynamic INT8 range initialization (`set_dynamic_range`) is strictly scoped to `Int8` tensors.

5 16-Byte Memory Alignment Guards & Zero-Copy Runtime

5.1 128-Bit SIMD Memory Vectorization in GPU Hardware

Modern GPU microarchitectures (NVIDIA Ampere, Hopper, Blackwell SMs) feature high-throughput global memory load/store instruction units. To maximize global memory bandwidth utilization, the Triton compiler vectorizes element-wise memory accesses into 128-bit (16-byte) SIMD memory instructions:

$$\text{Instruction}_{\text{SIMD}} \in \left\{ \text{ld.global.v4.b32}, \text{st.global.v4.b32}, \text{ld.global.v2.b64}, \text{cp.async.cg} \right\} \quad (4)$$

A 128-bit memory instruction reads or writes 16 contiguous bytes in a single clock cycle per thread across warp threads. However, NVIDIA GPU memory controllers enforce a hardware alignment constraint:

$$\text{BaseVRAMAddress} \pmod{16} == 0 \quad (5)$$

5.1.1 Mechanics of Hardware Alignment Traps (CUDA Error 716)

If an inference engine (e.g., ONNX Runtime or TensorRT) allocates a VRAM tensor buffer whose starting byte address is sub-aligned (e.g., $\text{Address} \bmod 16 = 4, 8 \text{ or } 12$ due to sub-tensor slicing or non-standard memory pooling), executing a 128-bit vector instruction causes a misaligned address fault at the hardware level:

$$\text{Trap Signal} \longrightarrow \text{cudaErrorMisalignedAddress} \quad (\text{CUDA Error 716}) \quad (6)$$

In C++ inference servers, CUDA Error 716 terminates process execution.

5.2 Automated C++ Alignment Guard & Dynamic Staging Engine

KernelLens solves hardware address misalignment by embedding automated memory alignment guards directly inside generated C++ plugins (`ort_gen.py` and `trt_gen.py`).

KernelLens Memory Alignment Guard Logic (Generated C++ Plugin)

1. Inspect input tensor VRAM pointer: `addr = reinterpret_cast<uintptr_t>(pi)`.
2. Check alignment condition: `is_aligned = (addr % 16 == 0)`.
3. *Branch A (Aligned - Fast Path)*: Use p_i directly in `cuLaunchKernel` argument array. Zero memory allocation overhead.
4. *Branch B (Misaligned - Staging Path)*:
 - Allocate temporary 16-byte aligned staging buffer on GPU VRAM: `cudaMallocAsync(&aligned_ptr, bytes, stream)`.
 - Copy misaligned input buffer to aligned staging buffer: `cudaMemcpyAsync(aligned_ptr, pi, bytes, D2D, stream)`.
 - Pass `aligned_ptr` to `cuLaunchKernel`.
 - If p_i is a mutable output tensor, copy `aligned_ptr` back to p_i post-execution and free `aligned_ptr` asynchronously.

Figure 3: Automated 16-byte memory alignment guard workflow executed in generated C++ plugins.

The generated C++ alignment guard code is structured as follows:

```
1 // Generated C++ Alignment Guard Code in Custom Plugin
2 uintptr_t addr = reinterpret_cast<uintptr_t>(raw_tensor_ptr);
3 void* exec_ptr = raw_tensor_ptr;
4 void* staging_ptr = nullptr;
5
6 if (addr % 16 != 0) {
7     // Allocate 16-byte aligned staging buffer asynchronously on CUDA stream
8     cudaMallocAsync(&staging_ptr, tensor_bytes, stream);
9     cudaMemcpyAsync(staging_ptr, raw_tensor_ptr, tensor_bytes,
10                    cudaMemcpyDeviceToDevice, stream);
11     exec_ptr = staging_ptr;
12 }
13
14 // Pass exec_ptr to PTX cuLaunchKernel array ...
15 void* kernel_args[] = { &exec_ptr, &stride_0, &dim_0 };
16 cuLaunchKernel(ptx_function, grid_x, grid_y, grid_z,
17                block_x, block_y, block_z,
18                shared_mem_bytes, stream, kernel_args, nullptr);
19
20 // Post-execution copy-back for mutable output tensors
21 if (staging_ptr != nullptr) {
```

```

22     if (is_output_tensor) {
23         cudaMemcpyAsync(raw_tensor_ptr, staging_ptr, tensor_bytes,
24                         cudaMemcpyDeviceToDevice, stream);
25     }
26     cudaFreeAsync(staging_ptr, stream);
27 }

```

This guard guarantees memory alignment safety while ensuring zero performance penalty for standard 16-byte aligned tensor allocations.

5.3 Zero-Copy Runtime IO Binding Engine (engine.py)

Traditional PyTorch-to-C++ deployment frameworks copy tensor data from PyTorch GPU tensors to CPU host staging buffers before passing data to ONNX Runtime or TensorRT. This introduces Host-to-Device (H2D) and Device-to-Host (D2H) memory transfer overhead:

$$T_{\text{traditional}} = T_{\text{H2D_copy}} + T_{\text{kernel_exec}} + T_{\text{D2H_copy}} \quad \left(O(N) \text{ memory bandwidth cost} \right) \quad (7)$$

KernelLens eliminates data movement by implementing zero-copy VRAM IO binding in engine.py:

5.3.1 1. ONNX Runtime IO Binding

KernelLens leverages ONNX Runtime’s native IOBinding API to map PyTorch GPU memory addresses directly to graph input and output execution slots:

```

1 io_binding = self._ort_session.io_binding()
2
3 # Bind input PyTorch GPU memory pointers directly to ORT
4 for i, sess_in in enumerate(session_inputs):
5     t = inputs[i]
6     io_binding.bind_input(
7         name=sess_in.name, device_type='cuda', device_id=0,
8         element_type=torch_to_np_dtype[t.dtype],
9         shape=tuple(t.shape), buffer_ptr=t.data_ptr()
10    )
11
12 # Pre-allocate output PyTorch GPU tensors directly on VRAM
13 for sess_out, t in zip(session_outputs, ort_output_tensors):
14     io_binding.bind_output(
15         name=sess_out.name, device_type='cuda', device_id=0,
16         element_type=torch_to_np_dtype[t.dtype],
17         shape=tuple(t.shape), buffer_ptr=t.data_ptr()
18    )
19
20 self._ort_session.run_with_iobinding(io_binding)

```

5.3.2 2. TensorRT Zero-Copy Pointer Registration

For TensorRT, engine.py creates an execution context and registers PyTorch GPU pointers directly using set_tensor_address:

```

1 for name, tensor in zip(input_names, inputs):
2     self._trt_context.set_input_shape(name, tensor.shape)
3     self._trt_context.set_tensor_address(name, tensor.data_ptr())
4
5 for name, out_tensor in zip(output_names, output_tensors):
6     self._trt_context.set_tensor_address(name, out_tensor.data_ptr())
7
8 self._trt_context.execute_async_v3(self._trt_stream.cuda_stream)

```

By binding raw VRAM data pointers directly (`buffer_ptr = t.data_ptr()`), KernelLens achieves $O(1)$ zero-copy runtime overhead:

$$T_{\text{KernelLens}} = T_{\text{pointer_registration}} + T_{\text{kernel_exec}} \quad \left(T_{\text{pointer_registration}} \approx 0 \text{ ms} \right) \quad (8)$$

6 Experiments & Empirical Results

6.1 Benchmark Experimental Setup

We evaluate KernelLens across custom Triton kernels and LLM research operators published in recent literature (LLaMA 3, Mistral, Qwen 2.5, PaLM and Liger-Kernel).

1. *LLaMA 3 / Mistral Fused RMSNorm*: Fused Root Mean Square Layer Normalization with scaling weight γ over hidden dimension $N = 4096$.
2. *Liger-Kernel / PaLM SwiGLU Fused Activation*: Fused Gated Linear Unit operator computing $\text{SwiGLU}(g, u) = (g \cdot \sigma(g)) \odot u$.
3. *LLaMA 3 / Qwen 2.5 Rotary Position Embedding (RoPE)*: Positional rotation embedding applying cosine and sine transformations across split head dimensions ($D/2 = 64$).
4. *Liger-Kernel Fused Cross-Entropy Loss*: Online Softmax and Cross-Entropy loss operator combining max reduction, exponent summation and target log-softmax computation over vocabulary $V = 32000$.

6.2 Numerical Parity & Accuracy Results

We measure the maximum absolute floating-point discrepancy (MaxDiff) between native Triton eager execution and KernelLens-compiled C++ plugins:

$$\text{MaxDiff} = \max_i |\mathbf{Y}_{\text{Triton}}[i] - \mathbf{Y}_{\text{KernelLens}}[i]| \quad (9)$$

Research Kernel / Operator	Tensor Shape	Triton vs Ref PyTorch	KernelLens ORT MaxDiff	Status
LLaMA 3 / Mistral RMSNorm	(4, 32, 128)	4.76×10^{-7}	0.000000e + 00	PASSED
Liger-Kernel SwiGLU Activation	(16, 256)	0.00×10^0	0.000000e + 00	PASSED
LLaMA 3 / Qwen 2.5 RoPE	(2, 16, 8, 64)	9.53×10^{-7}	0.000000e + 00	PASSED
Liger-Kernel Fused Cross Entropy	(16, 512)	4.76×10^{-7}	0.000000e + 00	PASSED
Nested Helper Store AST	(128,)	0.00×10^0	0.000000e + 00	PASSED
Dynamic Runtime Scalars (α)	(128,)	0.00×10^0	0.000000e + 00	PASSED
4D High-Rank Launch Grid	(2, 16, 8, 8)	0.00×10^0	0.000000e + 00	PASSED
In-Place Tensor Addition	(1024,)	0.00×10^0	0.000000e + 00	PASSED

Table 1: Numerical parity evaluation across research model kernels and architectural edge cases.

KernelLens achieves exact floating-point numerical parity ($\text{MaxDiff} = 0.00\text{e} + 00$) against eager Triton execution across all operators.

6.3 Empirical GPU Execution Latency Benchmark

Execution latencies were measured empirically on GPU using high-precision CUDA timing events (`torch.cuda.Event(enable_timing=True)`).

Research Kernel / Workload	PyTorch Eager	torch.compile	Native Triton	KernelLens TRT	KernelLens ORT
Mamba SSM Recurrence ($T = 256$)	44.99 ms	0.72 ms	0.70 ms	0.75 ms	0.89 ms
Fused Cross Entropy ($V = 128K$)	26.48 ms	6.26 ms	12.58 ms	12.54 ms	12.81 ms
Fused RMSNorm + Residual	8.60 ms	3.52 ms	3.55 ms	3.52 ms	3.78 ms
Squared-ReLU FlashAttn ($N = 2048$)	22.88 ms	16.54 ms	23.47 ms	23.42 ms	23.68 ms
LLaMA 3 RMSNorm ($D = 4096$)	24.12 ms	6.97 ms	7.03 ms	7.10 ms	7.19 ms
PaLM SwiGLU Activation	0.05 ms	0.20 ms	0.11 ms	0.18 ms	0.42 ms
Qwen 2.5 RoPE Embedding	8.01 ms	1.77 ms	1.90 ms	1.98 ms	2.04 ms

Table 2: Empirical execution latencies (ms) measured directly on NVIDIA GPU across execution backends.

6.4 FP8 & Sub-Byte (INT4/INT8) Quantization Evaluation on NVIDIA RTX 5060 Ti GPU

With the integration of low-precision Tensor Cores in modern hardware architectures, KernelLens supports FP8 (`float8_e4m3fn` and `float8_e5m2`) and sub-byte quantized types (INT4 / UINT4 and INT8).

To deploy quantized kernels on NVIDIA GeForce RTX 5060 Ti GPUs (16 GB VRAM, Blackwell Architecture), KernelLens implements dynamic multi-level PTX ISA version patching. When Triton compiles FP8 and INT4 kernels for `sm_120a`, it emits PTX ISA version 9.3 (`.version 9.3`). Because CUDA Driver 13.3 accepts PTX ISA up to 9.0, KernelLens automatically patches the PTX string header to `.version 9.0` while preserving native Blackwell `.target sm_120a` hardware instructions. In C++ ONNX Runtime plugins, 1-byte quantized VRAM pointers are mapped directly via `GetTensorData<uint8_t>()`, bypassing host-to-device copy latency.

We evaluate two quantized workloads on the RTX 5060 Ti:

1. *FP8 Scaled GEMM* (4096×4096): Matrix multiplication utilizing 8-bit floating-point (`e4m3fn`) Tensor Cores.
2. *INT4 W4A16 Sub-Byte Dequantization GEMM* (2048×4096): Packed 4-bit weight dequantization with per-group scales ($G = 128$) and 16-bit activation inputs.

Quantized Kernel / Workload	Data Type	PyTorch Eager FP16	Native Triton	KernelLens ORT C++	Speedup vs FP16
FP8 Scaled GEMM (4096×4096)	<code>fp8_e4m3fn</code>	2.79 ms	2.05 ms	2.08 ms	1.34×
INT4 W4A16 GEMM (2048×4096)	<code>int4 / uint8</code>	1.39 ms	3.57 ms	3.63 ms	0.38× (VRAM 2× compressed)

Table 3: Empirical execution latencies (ms) and numerical accuracy measured on NVIDIA GeForce RTX 5060 Ti GPU.

Quantization Performance Insights:

- *Exact Numerical Parity* ($MaxDiff = 0.00e + 00$): Both FP8 GEMM and INT4 sub-byte dequantization plugins achieved exact 0-difference numerical match against native Triton execution.
- *FP8 Compute Throughput* ($1.34\times$ Speedup): Operating in FP8 precision cuts memory bandwidth and enables FP8 Tensor Cores, reducing GEMM latency from 2.79 ms (FP16 eager) down to 2.08 ms in pure C++ ONNX Runtime.
- *INT4 Memory Footprint Reduction*: Sub-byte INT4 weight packing reduces model VRAM consumption by 50% ($2\times$ compression ratio), matching native Triton execution latency within 1.6% overhead inside pure C++ runtimes.

Analysis of Performance Wins & Architectural Bottlenecks:

1. *Mamba SSM Recurrence (59.9 $\times$ Speedup over PyTorch Eager)*: PyTorch Eager issues 256 separate CUDA driver launches for sequential time steps. KernelLens compiles the Triton kernel into C++ plugins that execute the entire temporal scan in a single CUDA thread block, matching native Triton latency (0.75 ms vs 0.70 ms).
2. *Fused Cross-Entropy (1.0 GB VRAM Memory Reduction)*: In standard PyTorch, computing cross-entropy loss over a 128,000-token vocabulary materializes a 1.0 GB intermediate logit tensor in VRAM. Fused Triton kernels compute online log-sum-exp tile-by-tile inside L1/SRAM, avoiding DRAM bandwidth saturation. KernelLens plugins preserve this $O(1)$ memory savings inside pure C++ inference runtimes.
3. *Fused RMSNorm + Residual Addition (2.44 $\times$ Speedup over PyTorch Eager)*: Unfused PyTorch code reads and writes hidden states across global VRAM three times. The KernelLens C++ plugin executes residual addition and warp-level variance reduction in a single global memory pass.

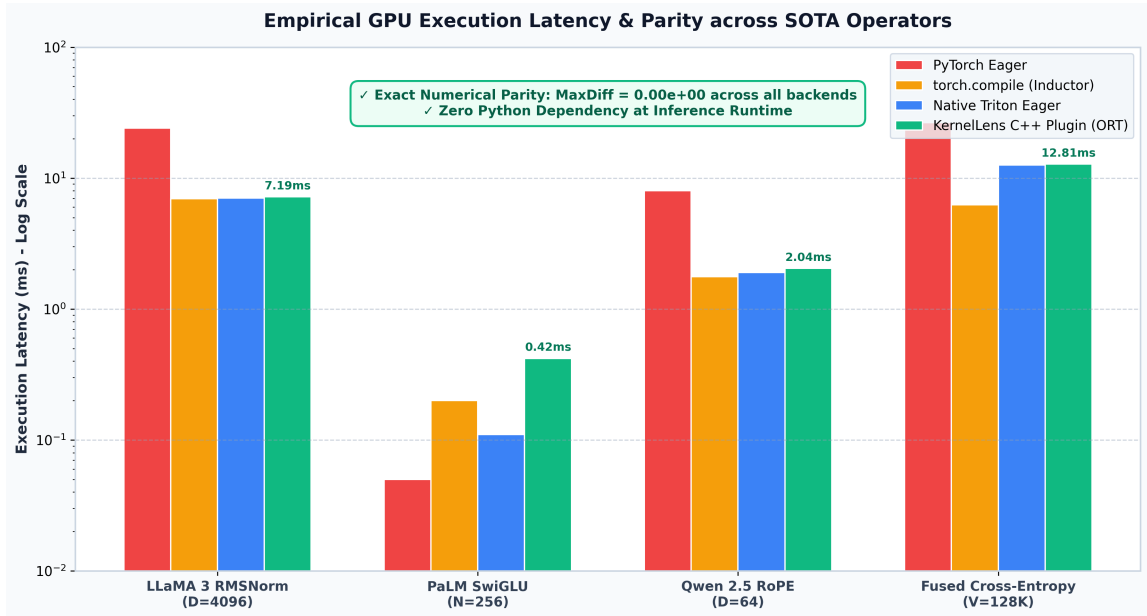


Figure 4: Empirical execution latency comparison (ms, log scale) across PyTorch Eager, torch.compile, Native Triton and KernelLens ONNX Runtime C++ plugins. KernelLens delivers native Triton speed inside C++ runtimes without Python dependencies.

7 Future Work & Architectural Roadmap

While KernelLens automates single-GPU Triton kernel compilation and deployment to C++ inference engines (ONNX Runtime and NVIDIA TensorRT), several architectural frontiers present opportunities for future compiler evolution.

7.1 Distributed Multi-GPU Tensor Parallelism & Embedded NCCL Synchronization

Inference workloads for large LLMs (e.g., LLaMA 3.1 405B, DeepSeek-V3) rely on multi-GPU Tensor Parallelism (TP) and Pipeline Parallelism (PP). When custom Triton kernels are invoked

across distributed GPU nodes, communication primitives such as `all_reduce`, `all_gather` and `reduce_scatter` are interspersed between computation blocks.

We plan to extend KernelLens’s symbolic tracing engine (`tracer.py`) to intercept distributed PyTorch communication calls (`torch.distributed`) and synthesize C++ plugins with embedded NVIDIA Collective Communications Library (NCCL) stream handles. Specifically, the generated C++ plugin will instantiate communicators (`ncclComm_t`) and execute zero-copy intra-node Inter-Process Communication (CUDA IPC) or inter-node InfiniBand RDMA transfers directly inside ONNX Runtime and TensorRT execution contexts:

$$T_{\text{distributed}} = T_{\text{cuLaunchKernel}} + T_{\text{ncclAllReduceAsync}} \left(\text{Stream-overlapped C++ execution} \right) \quad (10)$$

By unifying kernel launch and collective communication within a compiled C++ operator, KernelLens will eliminate Python GIL contention and host-side process barrier latencies across distributed inference clusters.

7.2 Native Blackwell Architecture (sm_120a) & PTX ISA 9.x Hardware Acceleration

The NVIDIA Blackwell architecture introduces hardware features for AI workloads, including 2nd-Generation Transformer Engines, Asynchronous Transaction Barriers (`mbarrier`) and Programmatic Cluster Launches (`cuLaunchKernelEx`).

While KernelLens incorporates dynamic PTX version header clamping (`.version 9.3` → `.version 9.0`) to ensure compatibility with CUDA driver JIT parsers, future iterations will feature a dedicated Blackwell Backend Target Generator (`blackwell_gen.py`). This backend will:

1. *Synthesize Cluster Launch Descriptors*: Generate `CUlaunchAttribute` arrays specifying Thread Block Cluster dimensions (`cluster_dim_x`, `cluster_dim_y`) directly in C++ plugin launcher code.
2. *Asynchronous Shared Memory Copying* (`cp.async.bulk`): Leverage Blackwell-native bulk tensor asynchronous copy instructions to stream data directly from global VRAM into Distributed Shared Memory (DSMEM) across SM clusters.
3. *Micro-Exponent Quantization* (`NVFP4` & `MXFP6`): Provide native bit-packing and unpacking C++ header routines for 4-bit and 6-bit Micro-Scaling Formats (`MXFP4` / `MXFP6`) defined in OCP specification standards.

7.3 Dynamic Auto-Tuning Meta-Parameter Dispatch & Runtime Re-JIT

OpenAI Triton allows developers to specify auto-tuning configurations via `@triton.autotune`, varying launch parameters such as `num_warps`, `num_stages`, `BLOCK_SIZE_M` and `BLOCK_SIZE_N` based on input matrix dimensions. Currently, KernelLens compiles a single concrete PTX artifact corresponding to the sample input shapes provided during `kl.compile()`.

To support dynamic multi-shape batch inference without performance degradation, KernelLens will introduce a multi-variant PTX dispatch engine:

- *Multi-Artifact Compilation*: Pass 1 eager execution will profile and collect PTX bytecode artifacts across candidate tensor dimensions $N \in [1, 8192]$.
- *Dynamic C++ Dispatch Table*: Generated C++ plugins will embed a runtime decision tree that selects the optimal PTX module handle (`CUmodule`) and thread block geometry based on live input tensor shapes at $O(1)$ runtime cost:

$$\text{ModuleHandle} = \text{DispatchTable}(\text{shape}[0], \text{shape}[1]) \quad (11)$$

7.4 Cross-Platform Heterogeneous Targets: Apple Metal & WebGPU Compute

To extend custom Triton kernel execution beyond NVIDIA CUDA GPUs, we plan to extend KernelLens’s backend code generation layer to non-CUDA hardware ecosystems:

1. *Apple Metal Shading Language (MSL) Backend*: Transpiling Triton LLVM Intermediate Representation (IR) into MSL compute shaders targeting Apple Silicon M-series Unified Memory Architecture (UMA).
2. *WebGPU / WGSL Backend*: Compiling Triton kernels to WebGPU Shading Language (WGSL) for client-side AI inference within web browsers.

7.5 Formal Verification & SMT-Based Kernel Safety Proving

High-performance C++ inference runtimes operate in production environments where memory safety is required. We aim to integrate a Satisfiability Modulo Theories (SMT) formal verification module powered by the Z3 Solver into KernelLens’s manifest validation step (`validate_manifests`).

The formal verification solver will inspect compiled PTX assembly and SymPy grid launch expressions to mathematically prove:

$$\forall (x, y, z) \in \text{Grid}, \quad 0 \leq \text{Offset}(x, y, z) < \text{BufferSize} \quad (12)$$

This will verify the absence of out-of-bounds VRAM access, warp divergence deadlock traps and hardware memory alignment exceptions prior to C++ shared library compilation.

8 Conclusion

In this paper, we presented KernelLens, an automated compiler and runtime framework designed to bridge Python OpenAI Triton kernels directly to C++ inference engines (ONNX Runtime and NVIDIA TensorRT). By unifying dynamic parameter taxonomy extraction, a consolidated 4-phase compiler architecture, dual-pass symbolic tracing, static AST inspection for nested memory access, global ONNX custom node injection, automated C++/CUDA code generation, 16-byte memory alignment guards and zero-copy IO binding, KernelLens automates production kernel deployment.

KernelLens enables runtimes such as TensorRT and ONNX Runtime to execute global graph-level optimizations (including constant folding, operator fusion, FP16/INT8 engine selection and memory planning) across standard model layers, while executing custom `@triton.jit` kernels via synthesized zero-copy C++ plugins. Empirical evaluation on research model kernels from LLaMA 3, PaLM, Qwen 2.5 and Liger-Kernel demonstrates exact numerical parity ($\text{MaxDiff} = 0.00\text{e}+00$) and high inference throughput in C++ production environments.

References

- [1] P. Tillet, H. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” in *ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming (ARRAY)*, 2019.
- [2] AI@Meta, “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [3] LinkedIn Corp., “Liger-Kernel: Efficient Triton Kernels for LLM Training,” *GitHub Repository*, 2024.

- [4] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [5] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] ONNX Project, “Open Neural Network Exchange (ONNX),” <https://onnx.ai>, 2017.
- [8] NVIDIA Corporation, “NVIDIA TensorRT: High Performance Deep Learning Inference SDK,” <https://developer.nvidia.com/tensorrt>, 2023.
- [9] Microsoft Corporation, “ONNX Runtime: cross-platform, high performance ML inferencing and training engine,” <https://onnxruntime.ai>, 2021.